

PARCIAL EMC 2025 FE

MANEJO DE ARCHIVOS Y LISTAS

Deconstrucción paso a paso. Código Limpio.

EL DESAFÍO

- Leer un archivo de origen (`prestamos.txt`).
- Procesar la información de la biblioteca.
- Generar un archivo nuevo por cada autor (`informe_[autor].txt`).

REGLAS ESTRUCTURADAS

- [✓] Solo Listas (Prohibido diccionarios, sets, tuplas).
- [✓] Cero POO (Programación Orientada a Objetos).
- [✓] Cero función `main` o `__main__`.
- [✓] Modularidad y Clean Code.

ANTES DEL CÓDIGO: MODOS DE APERTURA

Para interactuar con un archivo en Python, primero debemos "abrirlo". El modo de apertura le dicta al sistema operativo nuestra intención.





'r' - Read (Lectura)

- **Propósito:** Abre el archivo exclusivamente para **leer su contenido**.
- **Comportamiento:** Si el archivo no existe, Python arroja un **error crítico** (`FileNotFoundError`).
- **Protección:** No permite modificar ni **borrar** el contenido existente. Es el **modo seguro y por defecto**.



'w' - Write (Escritura)

- **Propósito:** Abre el archivo para escribir datos nuevos.
- **Comportamiento:** Si el archivo no existe, lo **crea automáticamente**.
- **Peligro:** Si el archivo ya existe, **Peligro:** Si el archivo ya existe, sobrescribe y **borra todo su contenido original** antes de escribir.

a

'a' - Append (Añadir)

- **Propósito:** Abre el archivo para agregar datos al final del mismo.
- **Comportamiento:** Si el archivo no existe, lo **crea**.
- **Protección:** Conserva intacto el texto previo y comienza a escribir exactamente después de la última línea.



'x' - Exclusive Creation

- **Propósito:** Crea un archivo nuevo estrictamente para escribir.
- **Comportamiento:** Falla categóricamente si el archivo ya existe (`FileExistsError`).
- **Uso:** Ideal para evitar sobrescribir archivos importantes por accidente.

NUESTRA ESTRATEGIA DE ACCESO

prestamos.txt (Origen)	prestamos.txt (Respaldo)	informe_[autor].txt (Destino)
Intentar leer con modo 'r' . Si falla, lo atraparemos.	Si falló la lectura, crearemos el archivo con modo 'w' y datos de ejemplo.	Generaremos reportes limpios desde cero usando modo 'w' .

ESTRUCTURA DEL ARCHIVO DE ORIGEN

Archivo de texto plano. Los datos están separados por el carácter punto y coma (;).

```
fecha; libro; autor; usuario;estado
```

```
2025-09-01; Rayuela; Julio Cortázar; Ana; devuelto
```

```
2025-09-02; Cien años; Gabriel G.; admin; pendiente
```

```
2025-09-03; El Extranjero; Albert Camus; Carlos; retrasado
```

EL RETO: SOLO LISTAS

Al no poder usar Diccionarios para mapear autores, ni Sets para aislar datos únicos, ni Objetos para estructurar la información, todo el ecosistema vivirá en una estructura primitiva, estricta y limpia:

LA LISTA DE LISTAS.

TRANSFORMACIÓN A MEMORIA (LISTA DE LISTAS)

`registros = [ ]`

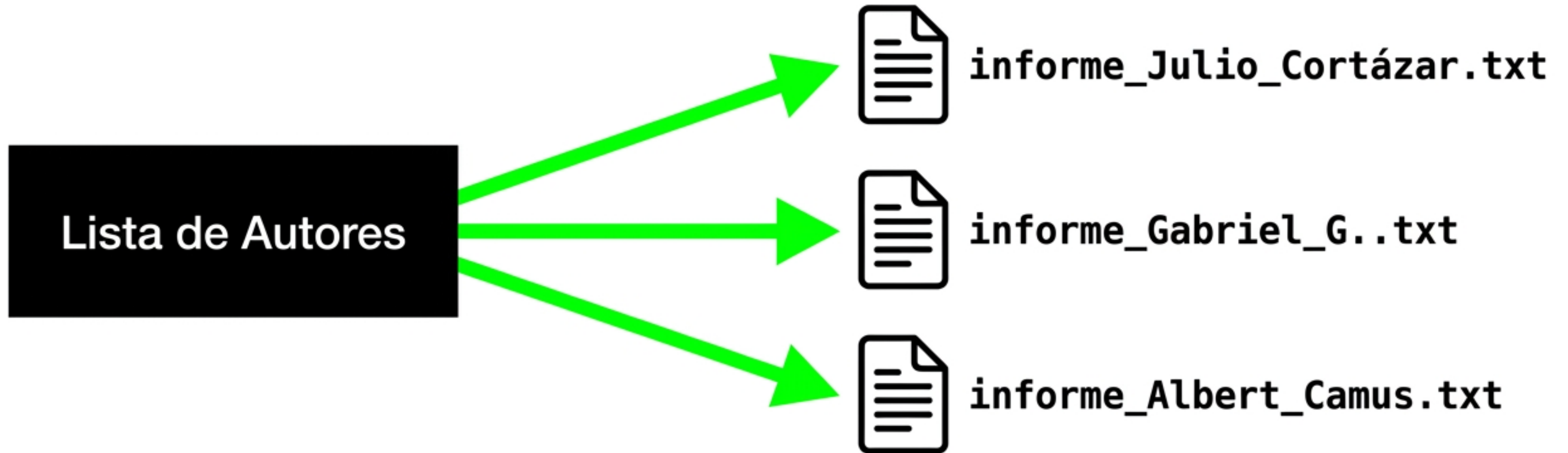
```
[ '2025-09-01', 'Rayuela', 'Julio Cortázar', 'Ana', 'devuelto' ]
```

```
[ '2025-09-02', 'Cien años', 'Gabriel G.', 'admin', 'pendiente' ]
```

Cada línea del archivo es una lista interna. Las columnas son índices fijos.
Ejemplo: **registro[2]** siempre será el Autor.

ARQUITECTURA DE SALIDA

Recorreremos los autores únicos y generaremos un archivo físico dedicado en el disco por cada uno.

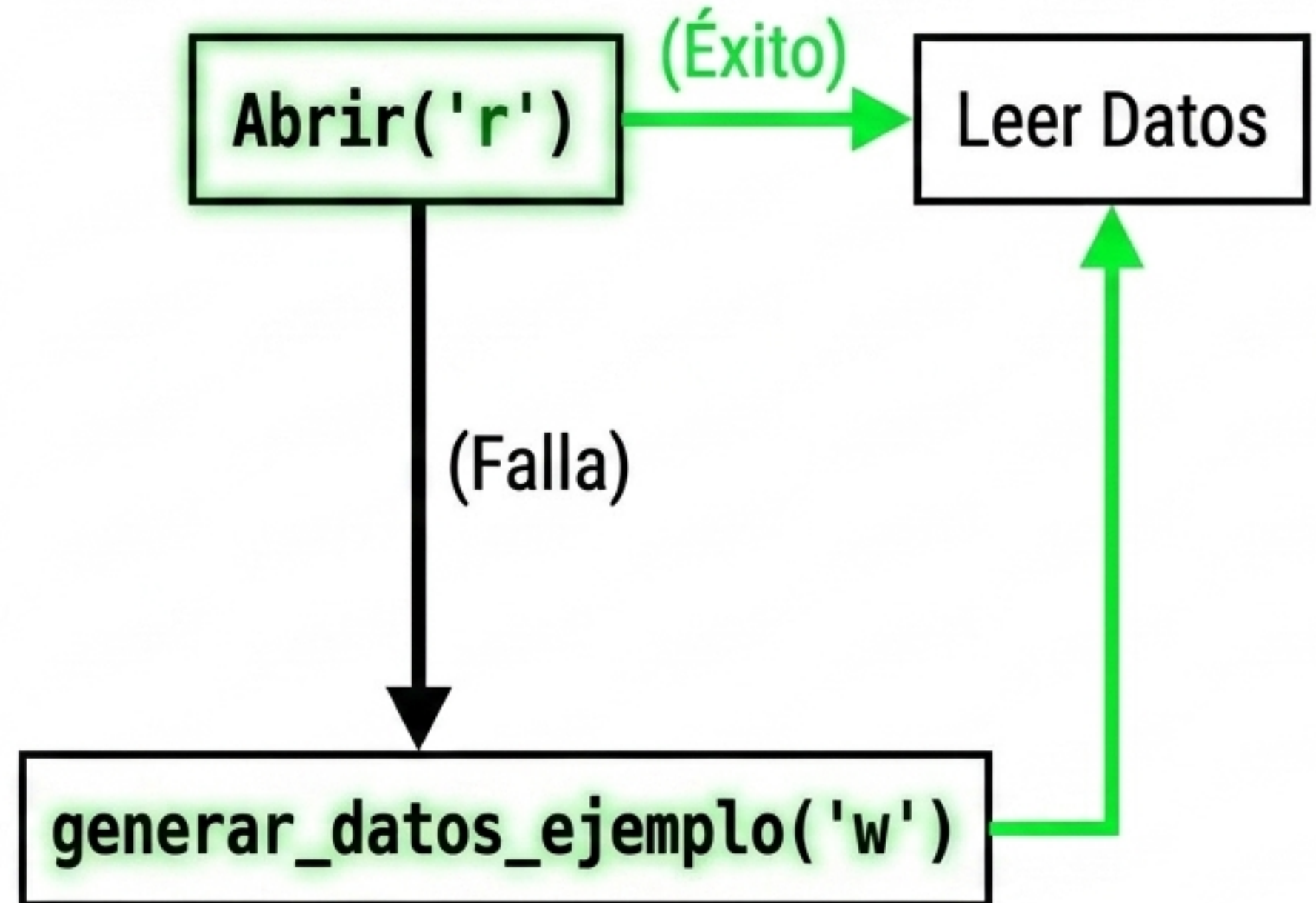


PASO 1: ASEGURAR LOS DATOS (LÓGICA)

El requerimiento exige: "si hay error debe crear el archivo con datos de ejemplo".

Usaremos un bloque **try / except**.

Si el sistema no encuentra el archivo al usar **'r'**, invocaremos una función de respaldo que use **'w'**.



PASO 1: FUNCIÓN DE RESPALDO (CÓDIGO)

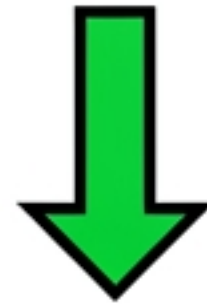
```
def generar_datos_ejemplo():  
    lineas = [  
        "2025-09-01; Rayuela; Julio Cortázar; Ana; devuelto\n",  
        "2025-09-02; Cien años de soledad; Gabriel García; admin; pendiente\n"  
    ]  
    archivo = open("prestamos.txt", "w")  
    for linea in lineas:  
        archivo.write(linea)  
    archivo.close()
```

————— Crea y escribe el
archivo desde cero.

PASO 2: LIMPIEZA DE CADENAS (LÓGICA)

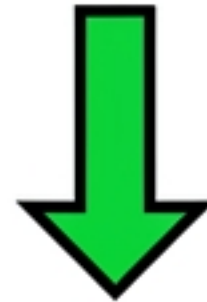
Extraemos la línea, quitamos saltos de línea y separamos por el delimitador.

```
"2025-09-01; Rayuela \n"
```



```
.strip()
```

```
"2025-09-01; Rayuela"
```



```
.split(';')
```

```
["2025-09-01", " Rayuela"]
```

PASO 2: EXTRACCIÓN DE DATOS (CÓDIGO I)

```
def leer_archivo_prestamos():  
    registros_finales = []  
  
    try:  
        archivo = open("prestamos.txt", "r")  
    except FileNotFoundError:  
        generar_datos_ejemplo()  
        archivo = open("prestamos.txt", "r")  
  
    # ... continua en la siguiente diapositiva ...
```

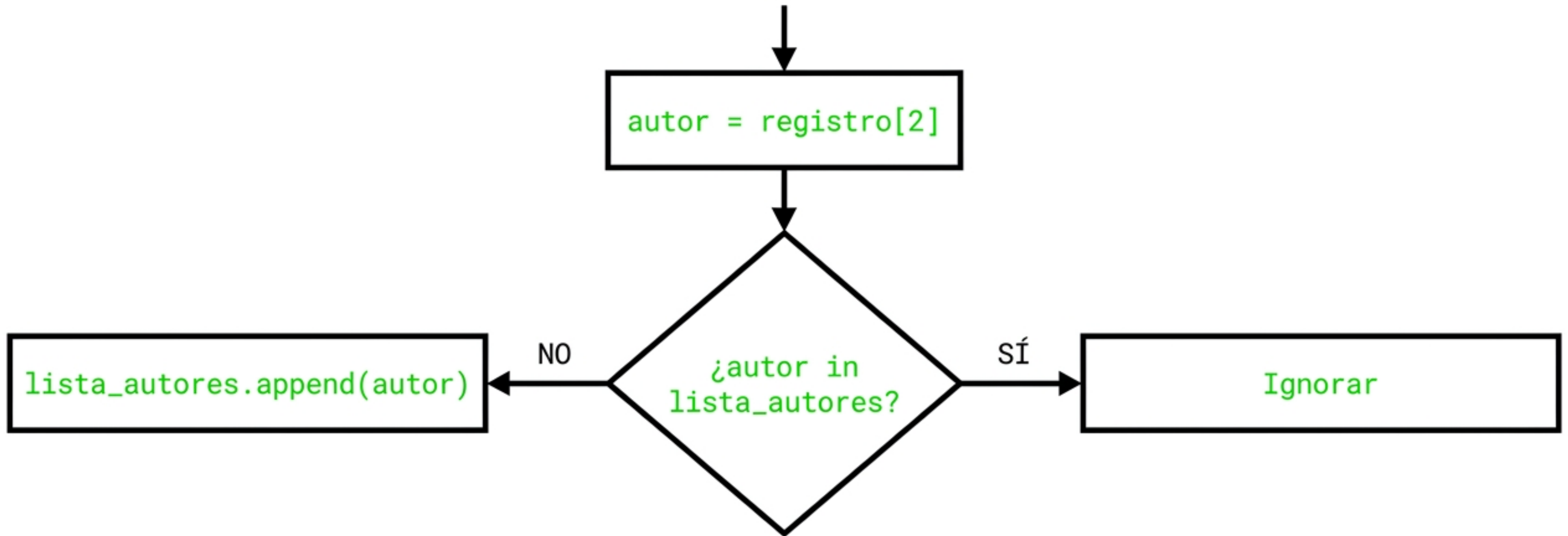
PASO 2: EXTRACCIÓN DE DATOS (CÓDIGO II)

```
for linea in archivo:  
    elementos = linea.strip().split(";")  
    registro_limpio = []  
  
    for elemento in elementos:  
        registro_limpio.append(elemento.strip())  
  
    registros_finales.append(registro_limpio)  
  
archivo.close()  
return registros_finales
```

Se retorna la matriz base purificada (Lista de Listas).

PASO 3: AUTORES ÚNICOS SIN SETS (LÓGICA)

Tradicionalmente usaríamos un **Set**. Bajo nuestra estricta restricción, iteramos buscando el índice [2] (Autor) y filtramos manualmente.



PASO 3: AUTORES ÚNICOS (CÓDIGO)

```
def obtener_autores_unicos(lista_registros):  
    autores = []  
  
    for registro in lista_registros:  
        autor = registro[2]  
  
        if autor not in autores:  
            autores.append(autor)  
  
    return autores
```

PASO 4: CRUZAR DATOS Y ESCRIBIR (LÓGICA)

1. Iterar sobre la lista de autores únicos.
2. Limpiar el nombre para el archivo (reemplazar espacios por guiones).
3. Abrir el archivo `informe_nombre.txt` en modo `'w'`.
4. Iterar sobre **todos** los registros: si el autor coincide, escribir la línea.
5. Cerrar el archivo.

PASO 4: CREACIÓN DE INFORMES (CÓDIGO I)

```
def generar_informes_por_autor(lista_registros, autores):  
    for autor in autores:  
        nombre_limpio = autor.replace(" ", "_")  
        nombre_archivo = "informe_" + nombre_limpio + ".txt"  
  
        archivo_salida = open(nombre_archivo, "w")  
        archivo_salida.write("=== INFORME DE: " + autor + " ===\n")  
  
    # ... continua en la siguiente diapositiva ...
```